振動波形デザインチュートリアル

2 つの正弦波バースト＋ピンクノイズ背景の刺激を作り、WAV と PNG に書き出す 5 ステップ

作成日: 2026-10-22 ／ 対象: python-vibrotactile チュートリアルを終えた人

`python-vibrotactile` では、1 本の長い正弦波を振動子から提示しました。実験で使う刺激は、短い振動（バースト）を決まった間隔で並べ、背景にノイズを重ねたものがよく使われます。このチュートリアルでは、**2 つの正弦波バースト**を **ISI（刺激間間隔）**を空けて並べ、背景に**ピンクノイズ**を重ねた波形を作り、**ISI・ノイズ振幅・パルス振幅**を変えながら **WAV ファイル**と**波形の画像（PNG）**に書き出します。

Step 1	`step1.py`	正弦波バーストを 1 つ作り、立ち上がり・立ち下がり（ランプ）をつける
Step 2	`step2.py`	バーストを 2 つ、ISI を空けて並べる
Step 3	`step3.py`	背景にピンクノイズを重ねる
Step 4	`step4.py`	完成した波形を WAV と PNG に書き出す
Step 5	`step5.py`	条件（ISI × パルス振幅 × ノイズ振幅）をまとめて書き出す

1. 準備と波形の全体像

ハードウェア構成（PC → オーディオ出力 → アンプ → 振動子）は `python-vibrotactile/tutorial.pdf` と同じです。ライブラリは前回の 2 つに加えて、グラフを描く `matplotlib` と WAV ファイルを書き出す `scipy` を使います。

```
pip install numpy sounddevice matplotlib scipy
```

最終的に作る波形は次の図のような構成です。ノイズは波形の最初から最後まで流れ続け、その上に 2 つのバーストが乗ります。

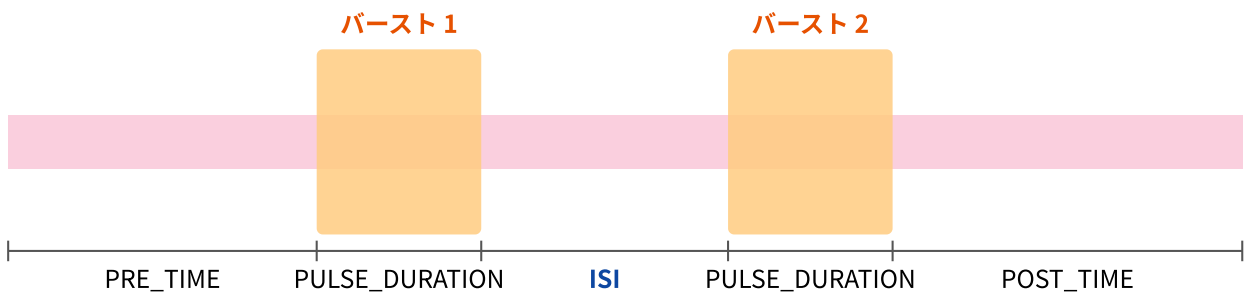


図 1. 完成する波形の構成。オレンジ = バースト（振幅 PULSE_AMPLITUDE）、ピンク = ピンクノイズ（振幅 NOISE_AMPLITUDE、全体に重ねる）

2. Step 1 — 正弦波バーストを 1 つ作る

バーストとは、決まった長さだけ続く短い振動です。`python-vibrotactile` の Step 1 と同じ式で正弦波を作り、長さを 0.1 秒にします。

step1.py

```
import numpy as np
import sounddevice as sd
import matplotlib.pyplot as plt

FREQUENCY = 250
PULSE_DURATION = 0.1
PULSE_AMPLITUDE = 0.5
RAMP = 0.01
SAMPLERATE = 44100

t = np.linspace(0, PULSE_DURATION, int(SAMPLERATE * PULSE_DURATION), endpoint=False)
pulse = PULSE_AMPLITUDE * np.sin(2 * np.pi * FREQUENCY * t)

n_ramp = int(SAMPLERATE * RAMP)
envelope = np.ones(len(pulse))
envelope[:n_ramp] = np.linspace(0, 1, n_ramp)
envelope[-n_ramp:] = np.linspace(1, 0, n_ramp)
pulse = (pulse * envelope).astype(np.float32)

sd.play(pulse, SAMPLERATE)
sd.wait()

plt.plot(t * 1000, pulse)
plt.xlabel("Time (ms)")
plt.ylabel("Amplitude")
plt.show()
```

プログラムの説明

大文字の変数が波形のデザインです。`PULSE_DURATION` はバーストの長さ（秒）、`PULSE_AMPLITUDE` は振幅（0.0〜1.0）、`RAMP` は立ち上がり・立ち下がりにかける時間（秒）です。

正弦波をいきなり始めたり止めたりすると、波形が急に変化する点で「プチッ」というクリック（衝撃的な振動）が生じます。そこで、0→1 に滑らかに上がって最後に 1→0 に下がる**包絡線** `envelope` を作り、正弦波に掛け算します。`np.ones` で全部 1 の配列を作り、先頭 `n_ramp` 個を `np.linspace(0, 1, n_ramp)`、末尾を `np.linspace(1, 0, n_ramp)` で上書きしています。

再生が終わると `matplotlib` の `plt.plot` で波形のグラフが表示されます。横軸は秒を 1000 倍してミリ秒にしています。

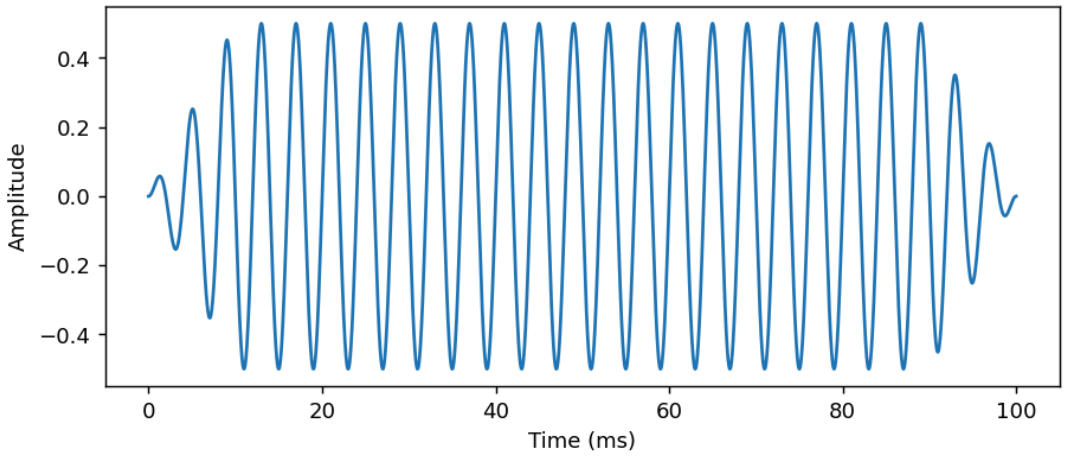


図 2. step1.py の出力（両端がランプで滑らかになっている）

試してみよう

`RAMP` を 0.001 と 0.02 で比べてみましょう。0.001 では始まりと終わりに「コツッ」という感触が加わります。`RAMP` は 0 より大きく、`PULSE_DURATION` の半分以下にしてください。

3. Step 2 — バーストを 2 つ、ISI を空けて並べる

同じバーストを 2 つ並べます。2 つの間に何も提示しない時間が **ISI（Inter-Stimulus Interval, 刺激間間隔）**です。

step2.py

```
import numpy as np
import sounddevice as sd
import matplotlib.pyplot as plt

FREQUENCY = 250
PULSE_DURATION = 0.1
PULSE_AMPLITUDE = 0.5
RAMP = 0.01
ISI = 0.2
SAMPLERATE = 44100

t = np.linspace(0, PULSE_DURATION, int(SAMPLERATE * PULSE_DURATION), endpoint=False)
pulse = PULSE_AMPLITUDE * np.sin(2 * np.pi * FREQUENCY * t)

n_ramp = int(SAMPLERATE * RAMP)
envelope = np.ones(len(pulse))
envelope[:n_ramp] = np.linspace(0, 1, n_ramp)
envelope[-n_ramp:] = np.linspace(1, 0, n_ramp)
pulse = pulse * envelope

gap = np.zeros(int(SAMPLERATE * ISI))
wave = np.concatenate([pulse, gap, pulse]).astype(np.float32)
time_axis = np.arange(len(wave)) / SAMPLERATE

sd.play(wave, SAMPLERATE)
sd.wait()

plt.plot(time_axis * 1000, wave)
plt.xlabel("Time (ms)")
plt.ylabel("Amplitude")
plt.show()
```

プログラムの説明

バーストを作るところまでは Step 1 と同じです。`np.zeros(int(SAMPLERATE * ISI))` で ISI 秒ぶんの無音（0 の配列）を作り、`np.concatenate([pulse, gap, pulse])` で「バースト → 無音 → バースト」の順につなげます。波形を「部品の配列をつなげて作る」という考え方は、この後のステップでも使います。

波形全体の長さが変わったので、グラフ用の時刻 `time_axis` はサンプル番号をサンプリング周波数で割って作り直しています。

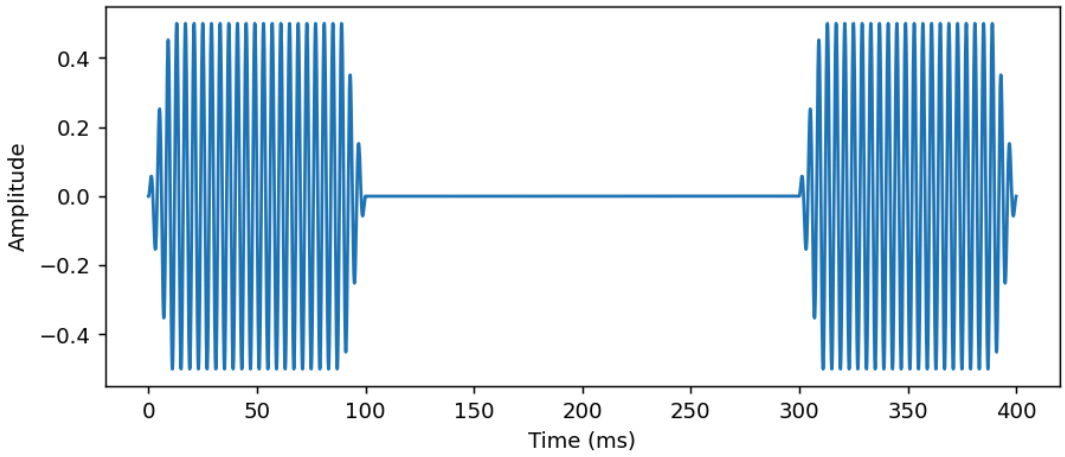


図 3. step2.py の出力（ISI = 200 ms）

ISI と SOA の違い: ISI は「1 つ目の終わり → 2 つ目の始まり」の時間です。「1 つ目の始まり → 2 つ目の始まり」は SOA（Stimulus Onset Asynchrony）と呼ばれ、SOA = `PULSE_DURATION` + ISI です。論文を読むときや書くときは、どちらの意味か確認しましょう。

試してみよう

`ISI` を 0.2 → 0.05 → 0.02 → 0.01 と短くしていき、2 つの振動が「1 つ」に感じられ始める間隔を探してみましょう。

4. Step 3 — 背景にピンクノイズを重ねる

バーストの前後に無音の時間を足し、全体にピンクノイズを重ねます。ノイズは、刺激を見つけにくくする「マスキング」や、実験環境の振動を模擬するために使われます。

step3.py

```
import numpy as np
import sounddevice as sd
import matplotlib.pyplot as plt

FREQUENCY = 250
PULSE_DURATION = 0.1
PULSE_AMPLITUDE = 0.5
RAMP = 0.01
ISI = 0.2
PRE_TIME = 0.5
POST_TIME = 0.5
NOISE_AMPLITUDE = 0.2
NOISE_SEED = 0
SAMPLERATE = 44100

t = np.linspace(0, PULSE_DURATION, int(SAMPLERATE * PULSE_DURATION), endpoint=False)
pulse = PULSE_AMPLITUDE * np.sin(2 * np.pi * FREQUENCY * t)

n_ramp = int(SAMPLERATE * RAMP)
envelope = np.ones(len(pulse))
envelope[:n_ramp] = np.linspace(0, 1, n_ramp)
envelope[-n_ramp:] = np.linspace(1, 0, n_ramp)
pulse = pulse * envelope

pre = np.zeros(int(SAMPLERATE * PRE_TIME))
gap = np.zeros(int(SAMPLERATE * ISI))
post = np.zeros(int(SAMPLERATE * POST_TIME))
bursts = np.concatenate([pre, pulse, gap, pulse, post])

rng = np.random.default_rng(NOISE_SEED)
white = rng.standard_normal(len(bursts))
spectrum = np.fft.rfft(white)
freqs = np.fft.rfftfreq(len(bursts), 1 / SAMPLERATE)
spectrum[0] = 0
spectrum[1:] = spectrum[1:] / np.sqrt(freqs[1:])
pink = np.fft.irfft(spectrum, len(bursts))
pink = pink / np.max(np.abs(pink))
noise = NOISE_AMPLITUDE * pink

wave = (bursts + noise).astype(np.float32)
time_axis = np.arange(len(wave)) / SAMPLERATE

if np.max(np.abs(wave)) > 1:
    print("Warning: the waveform exceeds 1.0 and will be clipped.")
```

```
sd.play(wave, SAMPLERATE)
sd.wait()

plt.subplot(2, 1, 1)
plt.plot(time_axis * 1000, wave)
plt.xlabel("Time (ms)")
plt.ylabel("Amplitude")
plt.subplot(2, 1, 2)
plt.loglog(freqs[1:], np.abs(np.fft.rfft(noise))[1:])
plt.xlabel("Frequency (Hz)")
plt.ylabel("Noise magnitude")
plt.tight_layout()
plt.show()
```

プログラムの説明

バースト列: `PRE_TIME` 秒の無音、バースト、ISI、バースト、`POST_TIME` 秒の無音をつなげて `bursts` を作ります。ノイズだけが流れる時間をバーストの前後に作るためです。

ピンクノイズ: ホワイトノイズはすべての周波数を同じ強さで含むノイズで、`rng.standard_normal`（正規乱数）で作れます。ピンクノイズは、周波数 f が 2 倍になるとパワーが半分になる（パワー ∝ 1/f）ノイズです。作り方は次の 3 段階です。

① `np.fft.rfft` でホワイトノイズを周波数成分 `spectrum` に分解する。② 各周波数成分を $\sqrt{f}$ で割る（振幅を $1/\sqrt{f}$ 倍するとパワーは 1/f 倍になる）。0 Hz（直流）は割れないので 0 にする。③ `np.fft.irfft` で時間波形に戻す。最後に最大値で割って振幅を ±1 に揃えてから `NOISE_AMPLITUDE` を掛けるので、`NOISE_AMPLITUDE` はノイズの**ピーク振幅**になります。

乱数の種: `np.random.default_rng(NOISE_SEED)` の `NOISE_SEED` を固定すると、何度実行しても同じノイズになります。条件間でノイズを揃えたい実験ではこの値を変えないでください。

クリップの確認: バーストとノイズを足した `wave` が ±1.0 を超えると、オーディオ出力で波形が切り取られ（クリップ）歪みます。超えた場合は警告を表示します。`PULSE_AMPLITUDE + NOISE_AMPLITUDE` が 1.0 以下なら超えません。

グラフは `plt.subplot(2, 1, 1)` ・ `(2, 1, 2)` で上下 2 段に分け、下段にノイズの周波数特性を両対数（`plt.loglog`）で描いています。右下がりの直線になっていれば 1/f のピンクノイズです。

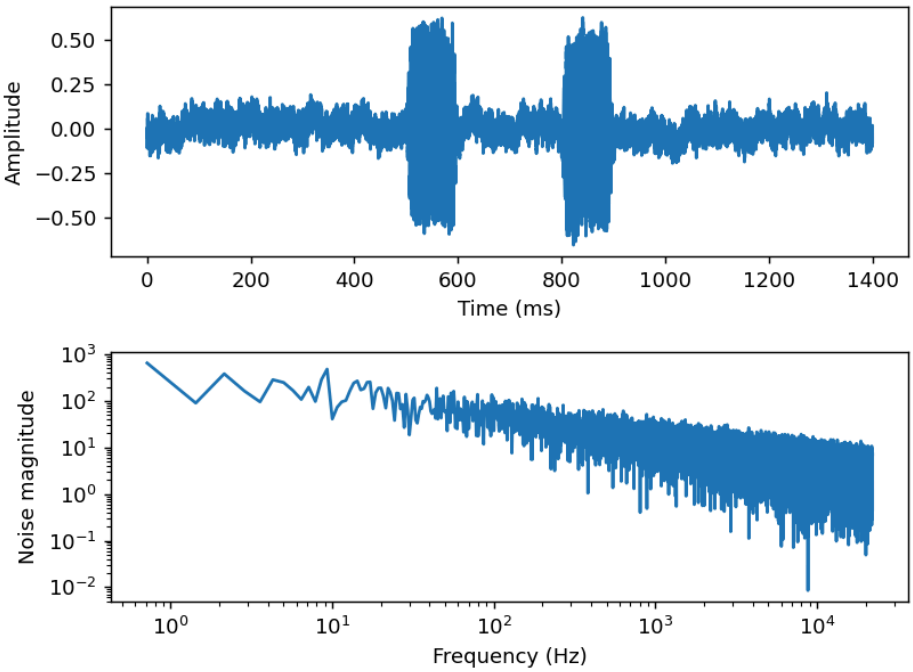


図 4. step3.py の出力（上: 波形、下: ノイズの周波数特性）

試してみよう

`NOISE_AMPLITUDE` を 0 → 0.1 → 0.3 と上げ、バーストがどれだけ感じにくくなるか比べてみましょう。

`spectrum[1:] / np.sqrt(freqs[1:])` の行を消すとホワイトノイズになるので、感触の違いも比べてみましょう。

5. Step 4 — WAV と PNG に書き出す

Step 3 の波形を、実験プログラムや他のソフトで使えるように WAV ファイルに、どんな波形かを記録するために PNG 画像に書き出します。

step4.py

```
import os
import numpy as np
import sounddevice as sd
import matplotlib.pyplot as plt
from scipy.io import wavfile

FREQUENCY = 250
PULSE_DURATION = 0.1
PULSE_AMPLITUDE = 0.5
RAMP = 0.01
ISI = 0.2
PRE_TIME = 0.5
POST_TIME = 0.5
NOISE_AMPLITUDE = 0.2
NOISE_SEED = 0
SAMPLERATE = 44100
OUTPUT_DIR = "output"

t = np.linspace(0, PULSE_DURATION, int(SAMPLERATE * PULSE_DURATION), endpoint=False)
pulse = PULSE_AMPLITUDE * np.sin(2 * np.pi * FREQUENCY * t)

n_ramp = int(SAMPLERATE * RAMP)
envelope = np.ones(len(pulse))
envelope[:n_ramp] = np.linspace(0, 1, n_ramp)
envelope[-n_ramp:] = np.linspace(1, 0, n_ramp)
pulse = pulse * envelope

pre = np.zeros(int(SAMPLERATE * PRE_TIME))
gap = np.zeros(int(SAMPLERATE * ISI))
post = np.zeros(int(SAMPLERATE * POST_TIME))
bursts = np.concatenate([pre, pulse, gap, pulse, post])

rng = np.random.default_rng(NOISE_SEED)
white = rng.standard_normal(len(bursts))
spectrum = np.fft.rfft(white)
freqs = np.fft.rfftfreq(len(bursts), 1 / SAMPLERATE)
spectrum[0] = 0
spectrum[1:] = spectrum[1:] / np.sqrt(freqs[1:])
pink = np.fft.irfft(spectrum, len(bursts))
pink = pink / np.max(np.abs(pink))
noise = NOISE_AMPLITUDE * pink

wave = (bursts + noise).astype(np.float32)
time_axis = np.arange(len(wave)) / SAMPLERATE
```

```
if np.max(np.abs(wave)) > 1:
    print("Warning: the waveform exceeds 1.0 and will be clipped.")

name = f"isi{ISI * 1000:.0f}ms_pulse{PULSE_AMPLITUDE}_noise{NOISE_AMPLITUDE}"
os.makedirs(OUTPUT_DIR, exist_ok=True)
wavfile.write(f"{OUTPUT_DIR}/{name}.wav", SAMPLERATE, wave)

onset_1 = PRE_TIME
onset_2 = PRE_TIME + PULSE_DURATION + ISI
plt.figure(figsize=(10, 4))
plt.plot(time_axis * 1000, wave, linewidth=0.5)
plt.axvspan(onset_1 * 1000, (onset_1 + PULSE_DURATION) * 1000, color="orange", alpha=0.2)
plt.axvspan(onset_2 * 1000, (onset_2 + PULSE_DURATION) * 1000, color="orange", alpha=0.2)
plt.title(f"{FREQUENCY} Hz bursts, ISI = {ISI * 1000:.0f} ms, "
          f"pulse amplitude = {PULSE_AMPLITUDE}, noise amplitude = {NOISE_AMPLITUDE}")
plt.xlabel("Time (ms)")
plt.ylabel("Amplitude")
plt.ylim(-1, 1)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/{name}.png", dpi=150)
print(f"Saved {OUTPUT_DIR}/{name}.wav and {OUTPUT_DIR}/{name}.png")

sd.play(wave, SAMPLERATE)
sd.wait()
plt.show()
```

プログラムの説明

波形を作るところまでは Step 3 と同じです。`OUTPUT_DIR` が書き出し先のフォルダで、`os.makedirs(..., exist_ok=True)` でフォルダがなければ作ります。

ファイル名: `name` に ISI・パルス振幅・ノイズ振幅を埋め込むので、例えば `isi200ms_pulse0.5_noise0.2.wav` のように、ファイル名を見ればどの条件の刺激か分かります。

WAV: `wavfile.write(パス, サンプリング周波数, 波形)` で書き出します。`float32` の配列をそのまま渡すと、−1.0〜1.0 の値をもつ 32 bit 浮動小数点の WAV になり、振幅の値がそのまま保存されます。

PNG: `plt.axvspan` でバーストの区間をオレンジ色に塗り、`plt.title` に条件を書き込みます。`plt.ylim(-1, 1)` で縦軸を固定しているので、条件の違う画像を並べても振幅を見比べられます。`plt.savefig` で保存してから、再生と画面表示を行います。

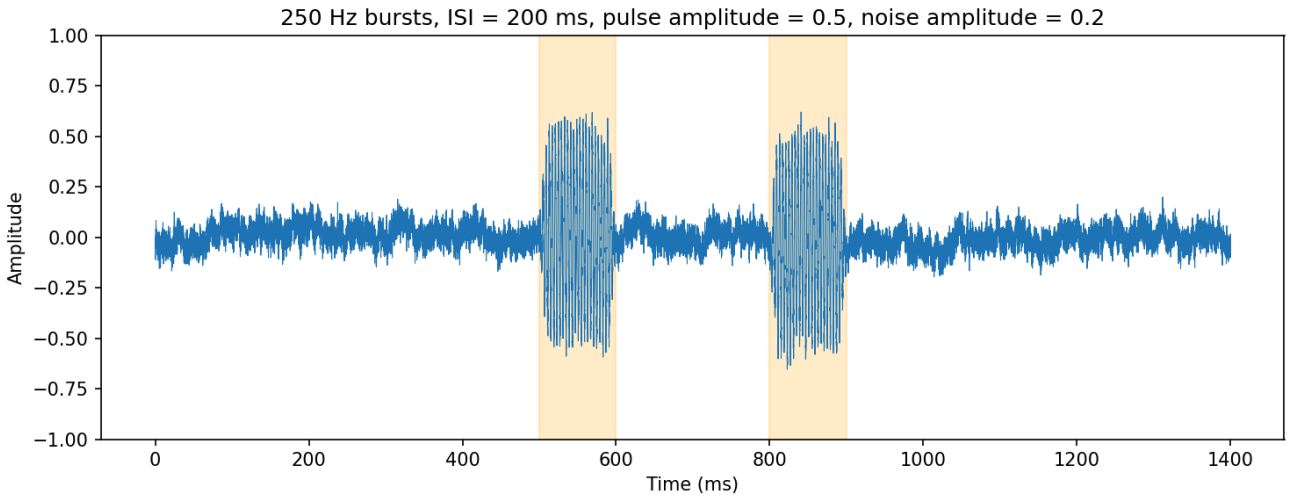


図 5. step4.py が書き出す PNG（output/isi200ms_pulse0.5_noise0.2.png）

試してみよう

`ISI`・`PULSE_AMPLITUDE`・`NOISE_AMPLITUDE` を変えて何回か実行し、`output` フォルダに条件ごとのファイルが増えていくことを確認しましょう。書き出した WAV は `python-vibrotactile` の `sd.play` でも再生できます。

6. Step 5 — 条件をまとめて書き出す

実験では複数の条件の刺激を用意します。Step 4 の処理を関数にまとめ、条件のリストを `for` 文で回して一度に書き出します。

step5.py

```
import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile

FREQUENCY = 250
PULSE_DURATION = 0.1
RAMP = 0.01
PRE_TIME = 0.5
POST_TIME = 0.5
NOISE_SEED = 0
SAMPLERATE = 44100
OUTPUT_DIR = "output"
ISI_LIST = [0.05, 0.1, 0.2]
PULSE_AMPLITUDE_LIST = [0.3, 0.5]
NOISE_AMPLITUDE_LIST = [0, 0.1, 0.2]


def make_wave(isi, pulse_amplitude, noise_amplitude):
    t = np.linspace(0, PULSE_DURATION, int(SAMPLERATE * PULSE_DURATION), endpoint=False)
    pulse = pulse_amplitude * np.sin(2 * np.pi * FREQUENCY * t)

    n_ramp = int(SAMPLERATE * RAMP)
    envelope = np.ones(len(pulse))
    envelope[:n_ramp] = np.linspace(0, 1, n_ramp)
    envelope[-n_ramp:] = np.linspace(1, 0, n_ramp)
    pulse = pulse * envelope

    pre = np.zeros(int(SAMPLERATE * PRE_TIME))
    gap = np.zeros(int(SAMPLERATE * isi))
    post = np.zeros(int(SAMPLERATE * POST_TIME))
    bursts = np.concatenate([pre, pulse, gap, pulse, post])

    rng = np.random.default_rng(NOISE_SEED)
    white = rng.standard_normal(len(bursts))
    spectrum = np.fft.rfft(white)
    freqs = np.fft.rfftfreq(len(bursts), 1 / SAMPLERATE)
    spectrum[0] = 0
    spectrum[1:] = spectrum[1:] / np.sqrt(freqs[1:])
    pink = np.fft.irfft(spectrum, len(bursts))
    pink = pink / np.max(np.abs(pink))

    return (bursts + noise_amplitude * pink).astype(np.float32)
```

```
os.makedirs(OUTPUT_DIR, exist_ok=True)
for isi in ISI_LIST:
    for pulse_amplitude in PULSE_AMPLITUDE_LIST:
        for noise_amplitude in NOISE_AMPLITUDE_LIST:
            wave = make_wave(isi, pulse_amplitude, noise_amplitude)
            if np.max(np.abs(wave)) > 1:
                print("Warning: the waveform exceeds 1.0 and will be clipped.")

            name = f"isi{isi * 1000:.0f}ms_pulse{pulse_amplitude}_noise{noise_amplitude}"
            wavfile.write(f"{OUTPUT_DIR}/{name}.wav", SAMPLERATE, wave)

            time_axis = np.arange(len(wave)) / SAMPLERATE
            plt.figure(figsize=(10, 4))
            plt.plot(time_axis * 1000, wave, linewidth=0.5)
            plt.title(name)
            plt.xlabel("Time (ms)")
            plt.ylabel("Amplitude")
            plt.ylim(-1, 1)
            plt.tight_layout()
            plt.savefig(f"{OUTPUT_DIR}/{name}.png", dpi=150)
            plt.close()
            print(f"Saved {name}")
```

プログラムの説明

Step 4 で 1 つずつ書き換えていた `ISI`・`PULSE_AMPLITUDE`・`NOISE_AMPLITUDE` を、`ISI_LIST`・`PULSE_AMPLITUDE_LIST`・`NOISE_AMPLITUDE_LIST` のリストにしました。関数 `make_wave(isi, pulse_amplitude, noise_amplitude)` は Step 4 の波形作成部分をそのまま関数にしたもので、3 つの値を受け取って波形を返します。

3 重の `for` 文がすべての組み合わせ（この例では 3 × 2 × 3 = 18 条件）を作り、それぞれ WAV と PNG を書き出します。たくさんの画像を作るので、`plt.close()` で描き終わった図を閉じてメモリを解放しています。再生はしません。

`make_wave` の中で毎回 `np.random.default_rng(NOISE_SEED)` を作り直しているため、ISI が同じ条件どうしではまったく同じノイズになります。ISI が違うと波形の長さが変わるため、ノイズの形も変わる点に注意してください。

試してみよう

`NOISE_AMPLITUDE_LIST` に 0.4 や 0.5 を加えて実行し、クリップの警告が出る条件を確認しましょう。`FREQUENCY` もリストにして、周波数の条件を増やしてみましょう。

7. トラブルシューティング

症状	確認すること
`ModuleNotFoundError`	`pip install numpy sounddevice matplotlib scipy` を実行したか
グラフのウィンドウが出たまま次に進まない	ウィンドウを閉じるとプログラムが終了します
`Warning: the waveform exceeds 1.0`	`PULSE_AMPLITUDE + NOISE_AMPLITUDE` を 1.0 以下にする
`ValueError`（ランプの部分）	`RAMP` が 0 より大きく、`PULSE_DURATION` の半分以下か
音（振動）が出ない	OS の既定出力デバイス／アンプの VOLUME・電源・ケーブル（python-vibrotactile と同じ）

⚠ 音量に関する注意
ノイズを重ねると、振幅のピークがバースト単体より大きくなります。アンプの VOLUME は最小から始め、振動を確認しながら少しずつ上げてください。